

Homework 7

Due: Friday, November 15th, 11pm

This worksheet contains only the *written* parts of HW7. The coding instructions will be released Thursday (11/14) and be due Monday (11/18). **The coding portion of this assignment is longer than in recent weeks, so you should aim to finish the written part early (Thursday night is a good goal) to have plenty of time to work on the code.**

Submission

After completing all parts below, submit your solutions as a PDF on Gradescope under “**HW7 Written**”. Don’t forget to check that the submitted file is up-to-date with all written work you completed!

Make sure your work is legible and scanned clearly if you handwrite it, or compiled correctly if you choose to use LaTeX. Match each HW problem to the page with your work when you turn in. If your work is not readable or pages are not assigned correctly, you will receive a point deduction.

Task 1 – List and Shout

[20 pts]

The following problems involve translation of loops into tail recursive functions.

1. The following TypeScript function calculates $\text{concat}(L, R)$ using a loop.

```
const concat = <A>(L: List<A>, R: List<A>): List<A> => {
  S = rev(L);
  while (S.kind !== "nil") {
    R = cons(S.hd, R);
    S = S.tl;
  }
  return R;
};
```

Define a mathematical definition for a tail-recursive function concat-acc that has identical behavior to the loop body. Then, give a mathematical definition of a function concat that calls concat-acc in such a manner that it matches the overall behavior of the concat code.

Don’t forget to add type declarations for your functions.

2. In HW5 Task 5, you defined a recursive function $f : \text{List}(\mathbb{Z}) \rightarrow \mathbb{Z}$ such that

$$f(L) := \text{sum}(\text{mult}(\text{zip}(L, \text{tail}(L)))) + \text{sum}(L)$$

The following function g calculates the same value using a loop:

```
const g = (L: List<bigint>): bigint => {
  if (L.kind === "nil")
    return 0;

  let s = 0;
  let w = L.hd;
  L = L.tl
  // Inv: f(L_0) = s + f(w :: L)
  while (L.kind !== "nil") {
    s = w * L.hd + w + s;
    w = L.hd;
    L = L.tl
  }
  return s + w;
};
```

While your recursive version (probably) worked by looking at the *first two* elements in the list to calculate the answer, this version keeps track of the previous element, before the part in L , in a separate variable w . That allows it to calculate the answer using the head of L and the previous element, w , rather than looking at first two elements of L .

This loop only makes sense, however, if the list is non-empty (so there is a first element). The code handles the case of a nil list in the first `if` statement. We only reach the loop when the list is not empty.

Define a tail-recursive function $g\text{-acc}$ that has identical behavior to the loop. Then, give a definition of g that matches the overall behavior of the function g , calling $g\text{-acc}$ in the cases where it would enter the loop and returning the answer directly in the other case(s).

Don't forget to add type declarations for your functions.

Task 2 – Loops, I Did it Again

[20 pts]

In HW6 Task 5, we looked at a function “even” that takes a list of base-3 digits, $\text{List}\langle\text{Digit}\rangle$, and determines if the value they represent is even or odd. Recall the definition of a Digit in base-3:

type Digit := 0 | 1 | 2

That function even was defined as follows:

```
even(nil)      := true
even(0 :: ds)  := even(ds)
even(1 :: ds)  := not even(ds)
even(2 :: ds)  := even(ds)
```

In this problem, we will do the same calculation using tail recursion. To do so, we first define a tail-recursive function, even-acc, of two arguments as follows:

```
even-acc(nil, b)    := b
even-acc(0 :: L, b) := even-acc(L, b)
even-acc(1 :: L, b) := even-acc(L, not b)
even-acc(2 :: L, b) := even-acc(L, b)
```

That would allow us to *re-define* even by invoking even-acc:

$\text{even}(L) := \text{even-acc}(L, \text{true})$

giving us a potentially more memory efficient implementation.

1. Translate the mathematical definitions for even-acc and the re-definition of even into one TypeScript function, $\text{even}(L)$, that behaves identically to a tail-call optimized version of these definitions by using a loop.

Be sure to include the invariant of the loop. Your invariant should be in the form shown in class for loops translated from tail recursion, and your code must be correct with that invariant.

2. Prove the following, which we'll refer to as “equation (1),” by induction

$$\text{even-acc}(L, b) = (\text{even}(L) = b) \tag{1}$$

where the “=” on the right is the usual equals operator on booleans, which is true if both have the same value and false if they have different values.

Note that $(b = \text{true})$ is equivalent to just b , and that $(\text{not } a = b)$ is equivalent to $(a = \text{not } b)$ since both mean $(a \neq b)$.

Hint: You will likely need a proof by cases within your inductive proof.

3. Using equation (1), which we proved holds in the last part, we can now rewrite our invariant without reference to even-acc (“Destroy the evidence”). This is useful to us because we can describe the behavior of our loop without reference to our intermediate tail-optimized version of the math definitions.

Prove by calculation that $\text{even}(L_0) = (\text{even}(L) = b)$ holds.

Task 3 – In One Fell Loop

[20 pts]

We have seen the following definition of a function $\text{contains}(L, y)$ that checks whether the value y appears in the list L :

$$\begin{aligned} \text{contains}(\text{List}\langle\mathbb{Z}\rangle, \mathbb{Z}) &\rightarrow \mathbb{B} \\ \text{contains}(\text{nil}, y) &:= \text{false} \\ \text{contains}(x :: L, y) &:= \text{true} \quad \text{if } x = y \\ \text{contains}(x :: L, y) &:= \text{contains}(L, y) \quad \text{if } x \neq y \end{aligned}$$

This function is already tail recursive.

1. Translate it into a TypeScript function, $\text{contains}(L, y)$, that behaves identically to a tail-call optimized version of these definitions by using a loop.

Be sure to include the invariant of the loop. Your invariant should be in the form shown in class for loops translated from tail recursion, and your code must be correct with that invariant.

2. It is also possible to define contains as follows

$$\begin{aligned} \text{cont}(\text{List}\langle\mathbb{Z}\rangle, \mathbb{Z}) &\rightarrow \mathbb{B} \\ \text{cont}(\text{nil}, y) &:= \text{false} \\ \text{cont}(x :: L, y) &:= (x = y) \text{ or } \text{cont}(L, y) \end{aligned}$$

This definition is more concise and uses pattern matching which may make it more obviously correct to someone else reading the code, but since it is not tail recursive, a direct implementation would be less memory efficient. We need to show that our tail-recursive function definition is equivalent to this alternate definition that we'd like to use to document the function instead.

Prove that the following, which we'll refer to as "equation (2),"

$$\text{cont}(L, y) = \text{contains}(L, y) \tag{2}$$

holds for all lists L and values y . Your proof should be by structural induction on L .

Note that, for all boolean values b , we have "false or $b = b$ " and "true or $b = \text{true}$ ".

3. Using equation (2), rewrite your loop invariant to use cont , instead of the original contains .

Task 4 – Extra Credit: 1, 2, 3, 4, I Declare a Num War

[0 pts]

The following function converts a natural number into a list of digits with base b .

$$\begin{aligned}\text{num-to-digits}(0, b) &:= \text{nil} \\ \text{num-to-digits}(n + 1, b) &:= ((n + 1) \bmod b) :: \text{num-to-digits}((n + 1) \text{ div } b)\end{aligned}$$

where “ $a \text{ div } b$ ” is integer division and “ $a \bmod b$ ” is remainder.

The “Division Equation” states that $a = (a \bmod b) + b \cdot (a \text{ div } b)$ holds for all natural numbers a and b . That, along with the fact that $0 \leq a \bmod b < b$, is essentially the *declarative* specification for the two functions, `mod` and `div`.

(a) We want to prove that the list returned by `num-to-digits( $n$ )` represent the number n , i.e., that

$$\text{value}(\text{num-to-digits}(n, b), b) = n$$

where `value` is the function that converts a list of digits in base $b \geq 2$ into a number:

$$\begin{aligned}\text{value}(\text{nil}, b) &:= 0 \\ \text{value}(x :: L, b) &:= x + b \cdot \text{value}(L, b)\end{aligned}$$

The function `num-to-digits` is not a simple structural recursion, for which a call on $n + 1$ would recurse on n . Instead, this recurses on $(n + 1) \text{ div } b$, which could be much smaller than n .

To do this proof, we need to use “strong” structural induction, where the inductive hypothesis assumes that the claim holds for all objects built in the process of producing $n + 1$. That includes all numbers from 0 up to n . For this proof, you are allowed to use that form of induction.

Suppose that we try to implement `num-to-digits` with the following loop.

```
/** @returns a list R such that value(R, b) = n */
const num_to_digits = (n: bigint, b: bigint): List => {
  let digits: List = nil;
  // Inv: rev(num-to-digits(n_0, b)) = rev(num-to-digits(n, b)) ++ digits
  while (n !== 0) {
    digits = cons(n % b, digits); // n % b means n mod b
    n = n / b;                   // n / b means n div b
  }
  return rev(digits);
};
```

The strange invariant for this loop is provided.

- (b) Prove that the loop invariant holds initially.
- (c) Prove that the body of the loop preserves the invariant.
- (d) Prove that the return value $R := \text{rev}(\text{digits})$ satisfies the specification “ $\text{value}(R, b) = n_0$ ”.

Note: you will need the fact proven in part 1. You will also need the fact that $\text{rev}(\text{rev}(L)) = L$ for any list L , a fact that we will call “Idempotency of `rev`”.

Homework 7

Due: Monday, November 18th, 11pm

If you have not completed HW7 written. Close this file and finish the written component before starting the coding parts in this worksheet.

To get started, check out the starter code for this assignment:

```
git clone https://gitlab.cs.washington.edu/cse331-24au-materials/hw7-calculator.git
```

Navigate to the hw7-calculator directory and run `npm install --no-audit`. Run tests with the command `npm run test` and run the linter with the command `npm run lint`. You can also run `npm run start` and open `localhost:8080` to see the application for this assignment. Currently, there will be unused variable Errors, but if you dismiss them with the 'X' you should be able to see the following app. The existing buttons do not yet work as expected, but they will after you implement the first task.

Base:

Digits:

The stack is empty... Push a number to get started.

Submission

After completing all tasks to follow, submit your solutions on Gradescope. The following completed files should be submitted to **"HW7 Code"**:

App.tsx

natural.ts

natural_test.ts

Wait after submitting to make sure the autograder passes, and leave yourself time to resubmit if there's an issue. The autograder will run your tests, additional staff tests, and the linter.

In the coding portion of this assignment, we will work with natural numbers represented as lists of digits, as in HW6 Task 5, but this time in an arbitrary base from 2 to 36. In the code, we represent a number as the following type:

```
type Natural = {digits: List<number>, base: number};
```

In addition to the types listed above, we require that base is between 2 and 36 (inclusive) and that every digit in the list is not just any number but specifically a number between 0 and base – 1 (inclusive).

When we write a base-10 number like “120”, the first digit “1” represents the 1×10^2 , so it is the “highest order” digit. Storing the high order digits at the front of a list is called “big endian” representation.

While the big endian representation matches how we write numbers, it is less convenient for our calculations, so instead, we will store the “lowest order” digits at the front of the list. For example, 120 would be stored as `0 :: 2 :: 1 :: nil` rather than `1 :: 2 :: 0 :: nil`. This is called a “little endian” representation, and it is what we will use in `Natural`.

We can formalize the discussion above by defining the value of the base- b digits in “ds” as

$$\begin{aligned}\text{value}(\text{nil}, b) &:= 0 \\ \text{value}(d :: \text{ds}, b) &:= d + b \cdot \text{value}(\text{ds})\end{aligned}$$

This simple, recursive definition encodes the fact that the digits are in the little endian representation. (A recursive function defining the value in the big endian representation would not be so simple!)

Note that we are not using `bigint` in this assignment. Instead, we have provided the above definition of `Natural` as an alternative. Also, note that we use `number` in the code to represent digits and bases, both of which should always be integers. In the starter code, we use some TypeScript functions that accept `number` as the parameter type, even though the inputs represent integers, so we use `number` everywhere for simplicity. You can assume digits and bases will be integers in your implementation. We won’t test your code on non-integer inputs.

Before continuing with the following tasks, read [these notes](#) on “bottom up” recursion which you will use for the loops in Task 1 and 2. These are also linked under [Topic 7](#).

Task 1 – Rally The Loops

[20 pts]

In this problem, we will implement functions that convert from strings to the `Natural` type above.

1. Implement the body of the function `naturalToString` in `natural.ts`. The function has the following declaration

```
const naturalToString = (nat: Natural): List<string>
```

Its specification says that, for all lists of digits `ds` (except `nil`, a special case), it returns the list of characters `rev(digits-to-str(nat))`, where `digits-to-str` is defined recursively as

```
digits-to-str(nil)      := nil
digits-to-str(d :: ds) := from-digit(d) :: digits-to-str(ds)
```

The reason for returning `rev(digits-to-str(nat))` rather than `digits-to-str(nat)` is that `digits-to-str` returns characters in little endian format, but we want to display them in big endian. The reversal switches between the two representations.

You should implement this function with a loop using the “**bottom up**” template.

Write down a correct invariant for your loop. Remember that we are not writing a loop to calculate `digits-to-str(nat)`, but rather `rev(digits-to-str(nat))`. The invariant must reflect this. Your code must be correct with the invariant you write! (It’s not enough to just behave properly when run, another programmer must be able to see why it is correct by reading your comments.)

A function that calculates `from-digit` is already provided in the code as `fromDigit`.

2. Write tests for `naturalToString` in `natural_test.ts`. Do not move on until you are certain that your code is correct. (Debugging it later on will be much more painful!) Write comments for your tests justifying which test coverage areas they satisfy.

We have provided the reverse of this operation, `stringToNatural`, which is an implementation of the following recursive definition:

```
str-to-digits(nil, b)      := nil
str-to-digits(c :: cs, b) := to-digit(c) :: str-to-digits(cs, b)  if to-digit(c) < b
str-to-digits(c :: cs, b) := undefined                             if to-digit(c) ≥ b
```

We implemented this function using recursion, but we could implement it with a loop using a template similar to the “**bottom up**” template.

Task 2 – Sticks Out Like a Sore Num

[20 pts]

In this problem, we will implement a function to add two natural numbers. This function has the following declaration in `natural.ts`:

```
const add = (nat: Natural, mat: Natural): Natural
```

This is the core part of our natural number library. The other functions, for multiplying and changing bases, are simple recursions that invoke this add function, provided in the starter code.

We will implement addition of digits as you learned in grade school, moving through the digits from the low order ones to the high order ones, bringing along a “carry” as we go. (This shows, again, why the little endian representation is easier for us.)

We can formalize the algorithm for adding two lists of digits in a fixed base B as follows:

$$\begin{aligned} \text{add}(\text{nil}, \text{nil}, 0) &:= \text{nil} \\ \text{add}(\text{nil}, \text{nil}, 1) &:= 1 :: \text{nil} \\ \text{add}(\text{nil}, b :: bs, c) &:= (b + c) :: bs && \text{if } b + c < B \\ \text{add}(\text{nil}, b :: bs, c) &:= (b + c - B) :: \text{add}(\text{nil}, bs, 1) && \text{if } b + c \geq B \\ \text{add}(a :: as, \text{nil}, c) &:= (a + c) :: as && \text{if } a + c < B \\ \text{add}(a :: as, \text{nil}, c) &:= (a + c - B) :: \text{add}(as, \text{nil}, 1) && \text{if } a + c \geq B \\ \text{add}(a :: as, b :: bs, 0) &:= (a + b + c) :: \text{add}(as, bs, 0) && \text{if } a + b + c < B \\ \text{add}(a :: as, b :: bs, c) &:= (a + b + c - B) :: \text{add}(as, bs, 1) && \text{if } a + b + c \geq B \end{aligned}$$

The first two and last two cases are the core of the algorithm. We need the middle cases to handle the fact that the lists of digits may not have the same length. We could handle this by adding some extra zeros to the end of the shorter list, but that is not necessary. The definition above handles those cases by behaving as if there was a 0 digit when one of the two becomes empty before the other.

Like the last task, this function follows the “**bottom up**” template. Make sure your code is correct with the provided invariant!

1. Implement the body of the loop so that it implements the function above in its recursive cases. As usual, we should only enter the loop body if the function needs to make a recursive call.
2. Implement the base cases of the function above after the loop, storing the result in the variable `rs` in the code. The code to put together the list from the base case with the list from the recursive cases, and to reverse the entire result, is already provided.
3. Write tests for `add` in `natural_test.ts`. Do not move on until you are certain that your code is correct. Write comments for your tests justifying which test coverage areas they satisfy.
4. Uncomment the tests for `scale`, `mul`, and `changeBase` in `natural_test.ts`. These should all pass now that `add` works. (If any of those failed, you’d have to figure out that code just to debug `add`. Ugh, can you imagine!)

Task 3 – Ahead of the Stack

[20 pts]

In this problem, we will finish the implementation of the stack calculator that uses your Natural numbers.

The code provided in `App.tsx` already allows the user to type in numbers, push them onto the stack, and to change the base of the numbers on the stack. If you haven't already, run `npm run start` to open up the app and see what you're working with, and read the provided `App.tsx` code to make sure you understand how state is managed and how the code is organized. The app is missing the ability to add and multiply numbers and to pop them from the stack which we will add in this part.

Note that the pop operation does not make sense when the stack is empty, and the add or multiply operations do not make sense when the stack has fewer than two items. When an operation does not make sense, you can either (1) not give the user a way to invoke it or (2) allow them to invoke it but show an error when the operation is not possible. You are free to choose whichever of these approaches you prefer. However, you cannot crash or silently fail when the operations do not make sense. That could be confusing to the user.

1. Implement the body of `renderStackOps` to return some HTML displaying buttons that allow the user to pop the top item from the stack, add the top two items, or multiply the top two items.
2. Implement event handlers for each of the buttons you created that, when clicked, perform the appropriate operations on the stack.
3. Manually test your UI to make sure that it properly invokes each of the operations.

We already know that each of the operations works properly, so we are just testing that the UI properly invokes them. (It's a good thing too. Can you imagine debugging a problem in the UI that was actually a bug in `add`. Yuck!)